

Association Rules

Part 1

Mohammed Brahimi & Sami Belkacem

Association Rules

- **Principle:** given a set of transactions, find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction.
- **Goal:** discovering interesting relationships hidden in large data sets.

Market-Basket transactions

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Juice, Eggs
3	Milk, Diaper, Juice, Coke
4	Bread, Milk, Diaper, Juice
5	Bread, Milk, Diaper, Coke

Example of Association Rules

$\{\text{Diaper}\} \rightarrow \{\text{Juice}\}$
 $\{\text{Milk, Bread}\} \rightarrow \{\text{Eggs, Coke}\}$
 $\{\text{Juice, Bread}\} \rightarrow \{\text{Milk}\}$

Implication means co-occurrence, not causality!

Note: Association Rules apply broadly but are illustrated with market basket data.

Examples of other application areas: bioinformatics, healthcare, web mining, etc.

Key Issues in Association Analysis

- **Challenges in Applying Association Rules:**

- Computational expense of discovering patterns in large datasets.
- Potential for false patterns and varying levels of pattern usefulness.

Slides Organization:

1. Basic concepts of association analysis.
2. Algorithms for efficient pattern mining:
 - a. Apriori
 - b. FP-Growth
3. Evaluating discovered patterns to avoid false results.

Outline

1. Frequent Itemsets and Association Rules
2. Apriori Algorithm
3. FP-Growth Algorithm
4. Evaluation of Association Patterns

Outline

- 1. Frequent Itemsets and Association Rules**
2. Apriori Algorithm
3. FP-Growth Algorithm
4. Evaluation of Association Patterns

Binary representation

- Market basket data can be represented in a binary format.
- A row corresponds to a transaction and a column corresponds to an item.
- An item is one if the item is present in a transaction and zero otherwise.
- An item is represented using a binary **asymmetric** variable.
- This representation ignore item quantities and purchase prices.

TID	Bread	Milk	Diapers	Juice	Eggs	Cola
1	1	1	0	0	0	0
2	1	0	1	1	1	0
3	0	1	1	1	0	1
4	1	1	1	1	0	0
5	1	1	1	0	0	1

Frequent Itemset

- **Itemset**

- A collection of one or more items
 - Example: {Milk, Bread, Diaper}
- k-itemset
 - An itemset that contains *k* items

- **Support count (σ)**

- Frequency of occurrence of an itemset
- E.g. $\sigma(\{\text{Milk, Bread, Diaper}\}) = 2$

- **Support (*s*)**

- Fraction of transactions that contain an itemset
- E.g. $s(\{\text{Milk, Bread, Diaper}\}) = 2/5$

- **Frequent Itemset**

- An itemset whose support is greater than or equal to a ***minsup*** threshold

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Juice, Eggs
3	Milk, Diaper, Juice, Coke
4	Bread, Milk, Diaper, Juice
5	Bread, Milk, Diaper, Coke

Association Rule

- **Association Rule**

- An implication expression of the form $X \rightarrow Y$, where X and Y are itemsets

- Example:

$$\{\text{Milk, Diaper}\} \rightarrow \{\text{Juice}\}$$

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Juice, Eggs
3	Milk, Diaper, Juice, Coke
4	Bread, Milk, Diaper, Juice
5	Bread, Milk, Diaper, Coke

- **Rule Evaluation Metrics**

- Support (**s**)

Fraction of transactions that contain both X and Y

- Confidence (**c**)

Measures how often items in Y appear in transactions that contain X

Example:

$$\{\text{Milk, Diaper}\} \Rightarrow \{\text{Juice}\}$$

$$s = \frac{\sigma(\text{Milk, Diaper, Juice})}{|T|} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk, Diaper, Juice})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67$$

Why Use Support and Confidence?

Support:

- Computes the frequency of a rule.
- Ensures rules are not due to chance (low support = low relevance).
- Helps identify profitable rules with user-defined thresholds.

Confidence:

- Computes the reliability of a rule.
- Higher confidence means the rule is more likely to hold true in new transactions.
- Indicates likelihood of **Y** given **X** (conditional probability).

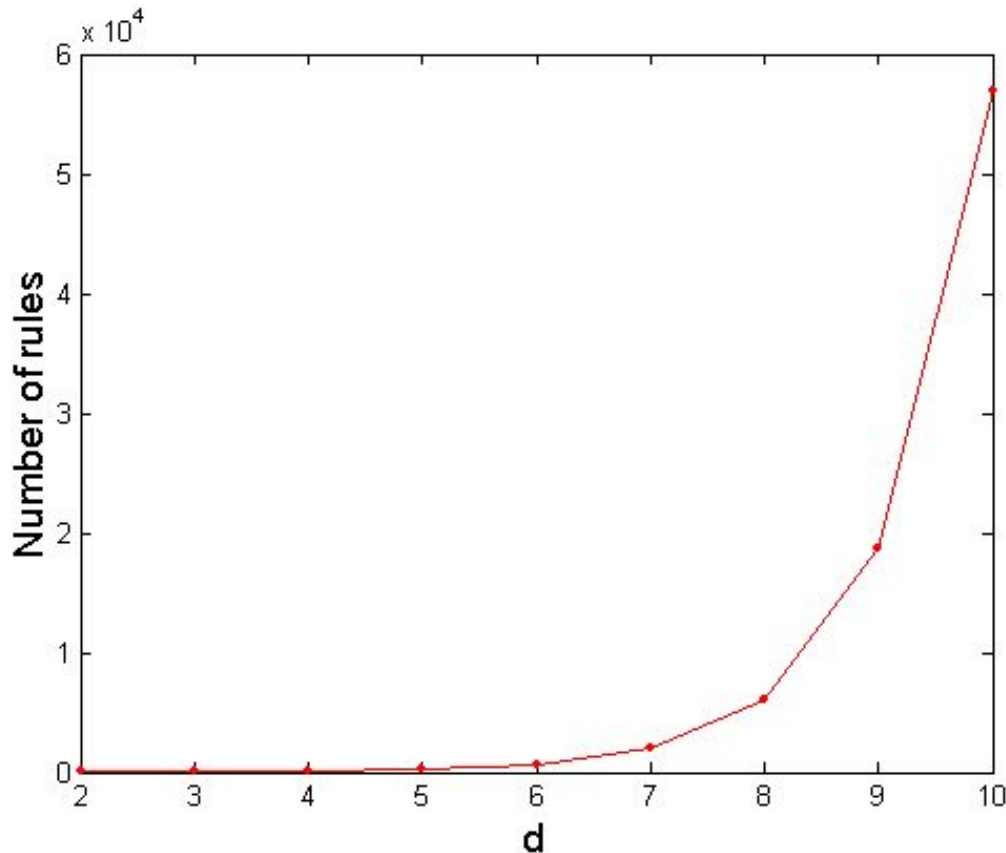
Association Rule Mining

- **Goal:** given a set of transactions T , the goal of association rule mining is to find all rules having
 - support $\geq$ ***minsup*** threshold.
 - confidence $\geq$ ***minconf*** threshold.
- **Approach:** one possible approach is brute-force.
 - List all possible association rules.
 - Compute the support and confidence for each rule.
 - Prune rules that fail the ***minsup*** and ***minconf*** thresholds.

⇒ Computationally prohibitive!

Computational Complexity

- Given d unique items:
 - Total number of itemsets = 2^d (including empty set)
 - Total number of possible association rules:



$$R = \sum_{k=1}^{d-1} \left[\binom{d}{k} \times \sum_{j=1}^{d-k} \binom{d-k}{j} \right]$$
$$= 3^d - 2^{d+1} + 1$$

If $d=6$, $R = 602$ rules

Mining Association Rules

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Juice, Eggs
3	Milk, Diaper, Juice, Coke
4	Bread, Milk, Diaper, Juice
5	Bread, Milk, Diaper, Coke

Example of Rules:

$\{\text{Milk, Diaper}\} \rightarrow \{\text{Juice}\}$ ($s=0.4$, $c=0.67$)
 $\{\text{Milk, Juice}\} \rightarrow \{\text{Diaper}\}$ ($s=0.4$, $c=1.0$)
 $\{\text{Diaper, Juice}\} \rightarrow \{\text{Milk}\}$ ($s=0.4$, $c=0.67$)
 $\{\text{Juice}\} \rightarrow \{\text{Milk, Diaper}\}$ ($s=0.4$, $c=0.67$)
 $\{\text{Diaper}\} \rightarrow \{\text{Milk, Juice}\}$ ($s=0.4$, $c=0.5$)
 $\{\text{Milk}\} \rightarrow \{\text{Diaper, Juice}\}$ ($s=0.4$, $c=0.5$)

Observations:

- All the rules are binary partitions of the same itemset: {Milk, Diaper, Juice}.
- Rules originating from the same itemset have identical support but can have different confidence.
- If the itemset is infrequent, then all six candidate rules can be pruned immediately without having to compute their confidence values.

Mining Association Rules

A common strategy of many algorithms is to break the problem into:

1. **Frequent Itemset Generation**

First generate all itemsets whose support $\geq$ *minsup*.

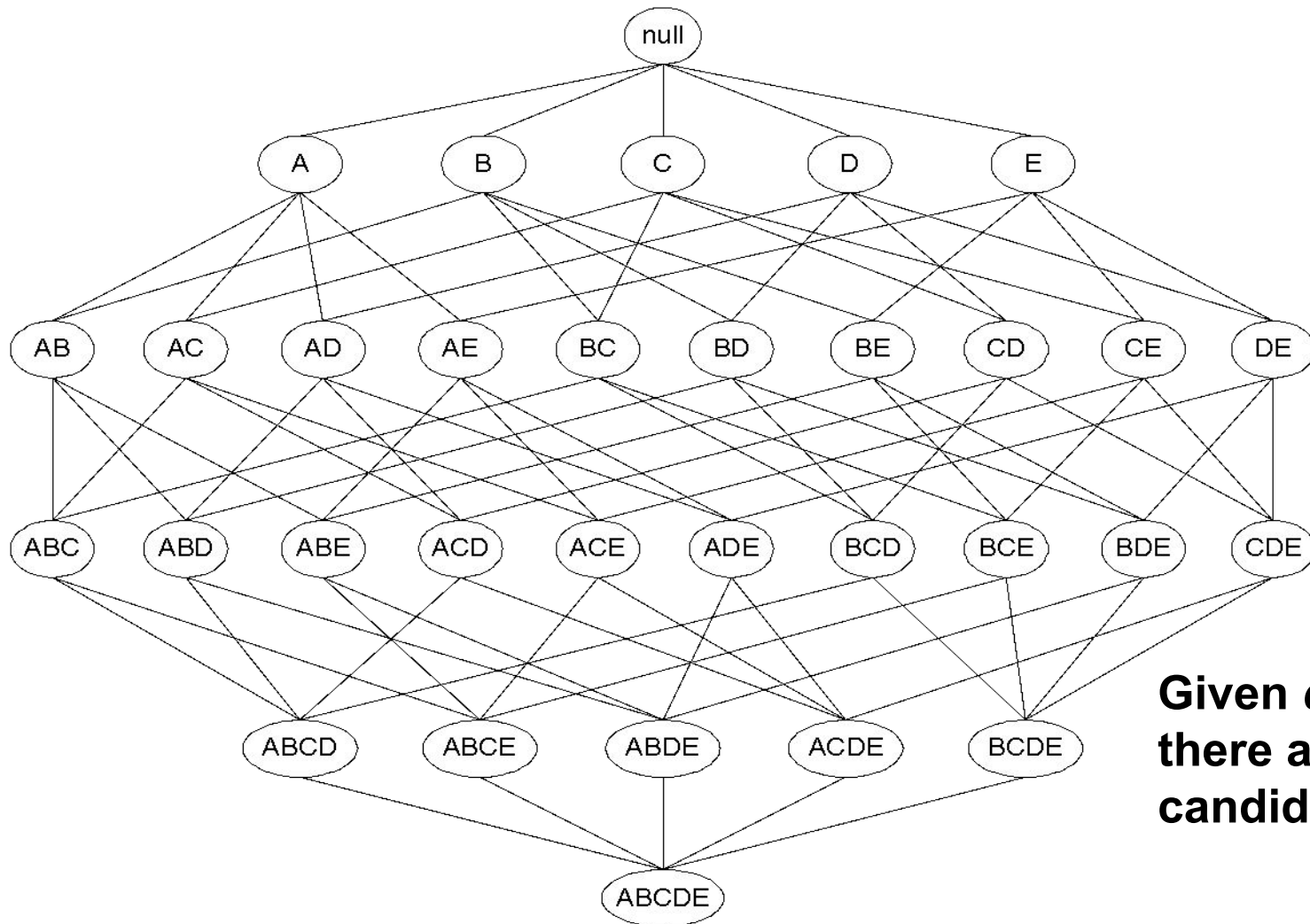
2. **Rule Generation**

Then generate high **confidence** rules from each frequent itemset, where each rule is a binary partitioning of a frequent itemset.

The computational requirements for frequent itemset generation are generally more expensive than those of rule generation.

Frequent Itemset Generation: Lattice with 5 items

A brute-force approach for finding frequent itemsets is to determine the support count for every candidate itemset in the lattice structure.

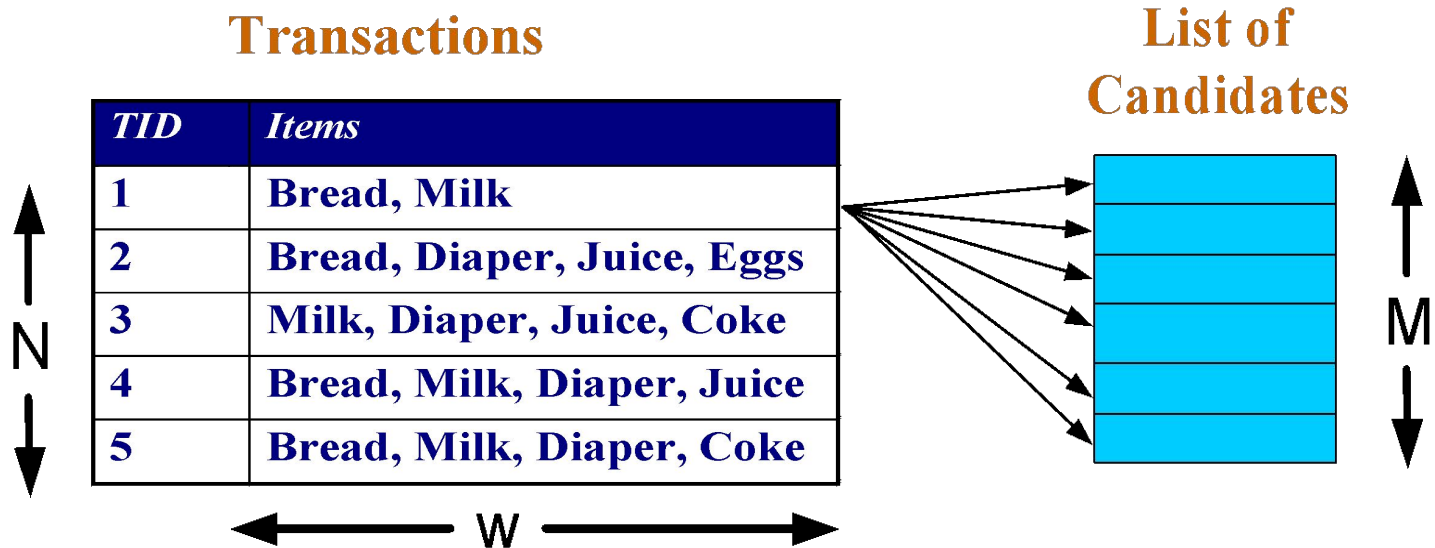


**Given d items,
there are 2^d possible
candidate itemsets**

Frequent Itemset Generation

- **Brute-force approach:**

- Each itemset in the lattice is a candidate frequent itemset.
- Count the support of each candidate by scanning the database.



- Match each transaction against every candidate in the lattice.
- **N**: # of transactions, **M**: # of candidate itemsets, **w**: maximum transaction width
- Complexity $\sim O(NMw) \Rightarrow$ Expensive since $M = 2^d$!!!

Frequent Itemset Generation Strategies

Three main approaches for reducing the computational complexity:

1. Reduce the **number of candidates** (**M**)
 - Complete search: $M=2^d$
 - Use pruning techniques to reduce **M**, e.g. using Apriori principle.
2. Reduce the **number of transactions** (**N**)
 - Reduce size of **N** as the size of itemset increases.
3. Reduce the **number of comparisons** (**NM**)
 - Use efficient data structures to store the candidates or transactions.
 - E.g. a hash structure
 - No need to match every candidate against every transaction.

Outline

1. Frequent Itemsets and Association Rules
- 2. Apriori Algorithm**
3. FP-Growth Algorithm
4. Evaluation of Association Patterns

Apriori principle

- Use **support** to limit candidate itemsets during frequent itemset generation.
- **Theorem:** If an itemset is frequent, then all of its subsets must also be frequent.

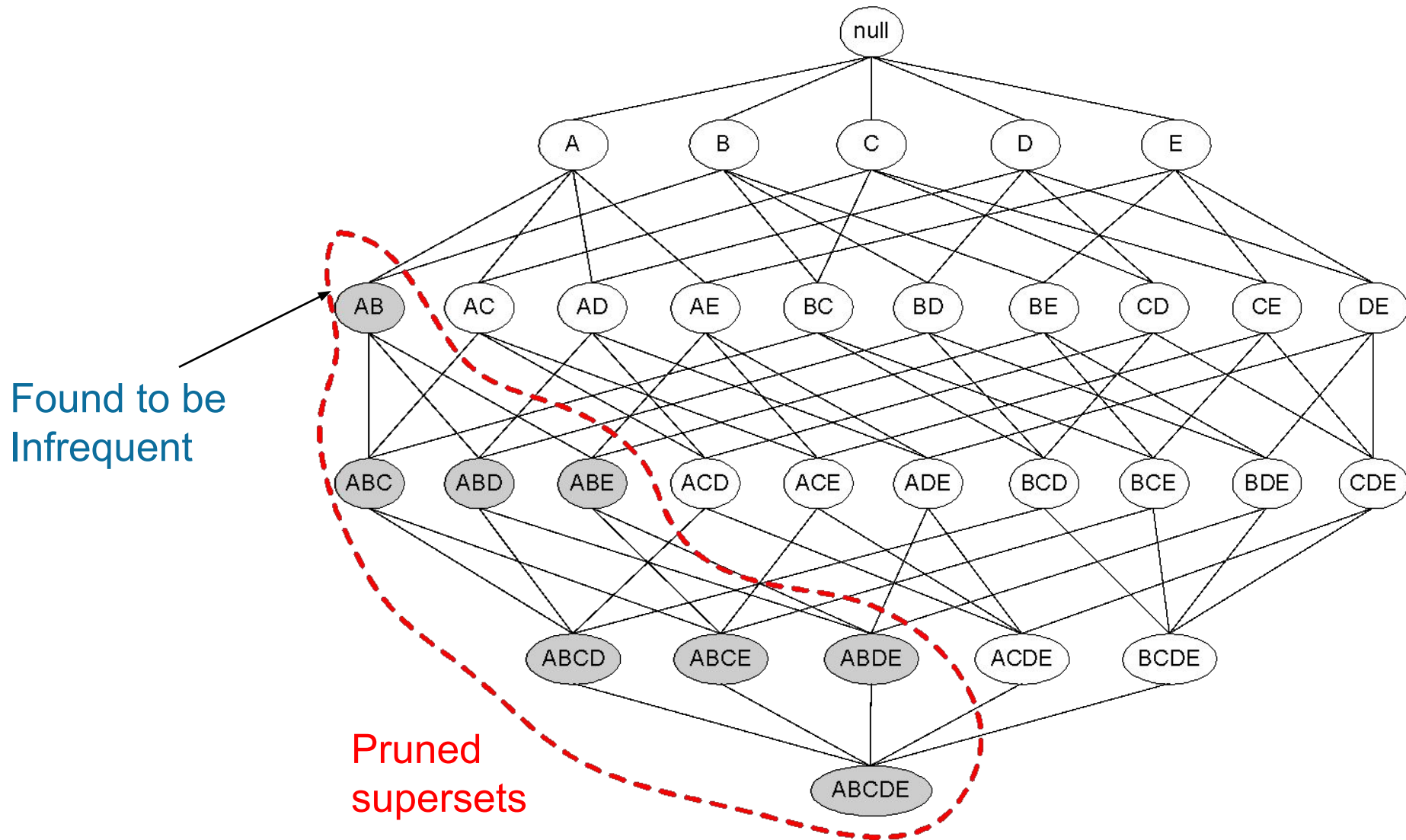
$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

- This theorem holds due to the following property of the **support** measure:

“Support of an itemset never exceeds the support of its subsets”

- Conversely, if an itemset is infrequent, then all itemsets that include this infrequent itemset should be infrequent.
- Use the latter principle to prune supersets of an infrequent itemset.

Illustrating Apriori Principle



Apriori Algorithm

Similar to all association rule algorithms, Apriori goes through the following steps:

- **Frequent Itemset Generation:**

Generate all itemsets whose support is greater than or equal to ***minsup***.

- **Rule Generation:**

Generate high-confidence rules from each frequent itemset, where the confidence is greater than or equal to ***minconf***.

In the upcoming slides, we will discuss each step in detail.

Frequent Itemset Generation

Initialization and Determination of Frequent 1-Itemsets:

Single pass to determine **support for each item**, resulting in frequent 1-itemsets (**F1**).

Iterative Generation of Candidate K-Itemsets:

Iteratively **generates** and **prunes candidate k-itemsets** based on previous iterations.

Counting Support of Candidates:

Additional pass to **count support for candidates** using the subset function.

Termination:

Algorithm stops when **no new frequent itemsets** are generated ($F_k = \emptyset$).

Frequent Itemset Generation - Illustrating Apriori Principle

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Juice, Bread, Diaper, Eggs
3	Juice, Coke, Diaper, Milk
4	Juice, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk



Items (1-itemsets)

Item	Count
Bread	4
Coke	2
Milk	4
Juice	3
Diaper	4
Eggs	1

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

nC_k : number of ways to choose k items from n items without considering the order
 ${}^nC_k = n! / (k! \times (n-k)!)$

Illustrating Apriori Principle

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Juice, Bread, Diaper, Eggs
3	Juice, Coke, Diaper, Milk
4	Juice, Bread, Diaper, Milk
5	Bread, Coke, Diaper, Milk



Items (1-itemsets)

Item	Count
Bread	4
Coke	2
Milk	4
Juice	3
Diaper	4
Eggs	1

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

Prune infrequent 1-itemset candidates

Illustrating Apriori Principle

Item	Count
Bread	4
Coke	2
Milk	4
Juice	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset
{Bread,Milk}
{Juice, Bread}
{Bread,Diaper}
{Juice,Milk}
{Diaper,Milk}
{Juice,Diaper}

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Generate 2-itemset candidates

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

Illustrating Apriori Principle

Item	Count
Bread	4
Coke	2
Milk	4
Juice	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread,Milk}	3
{Juice, Bread}	2
{Bread,Diaper}	3
{Juice,Milk}	2
{Diaper,Milk}	3
{Juice,Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

Prune infrequent 2-itemset candidates

Illustrating Apriori Principle

Item	Count
Bread	4
Coke	2
Milk	4
Juice	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread,Milk}	3
{Juice, Bread}	2
{Bread,Diaper}	3
{Juice,Milk}	2
{Diaper,Milk}	3
{Juice,Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$



Triplets (3-itemsets)

Itemset
{Juice, Diaper, Milk}
{Juice, Bread, Diaper}
{Bread, Diaper, Milk}
{Juice, Bread, Milk}

Generate 3-itemset candidates

Illustrating Apriori Principle

Item	Count
Bread	4
Coke	2
Milk	4
Juice	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread,Milk}	3
{Juice, Bread}	2
{Bread,Diaper}	3
{Juice,Milk}	2
{Diaper,Milk}	3
{Juice,Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$



Triplets (3-itemsets)

Itemset	Count
{Juice, Diaper, Milk}	2
{Juice, Bread, Diaper}	2
{Bread, Diaper, Milk}	2
{Juice, Bread, Milk}	1

Prune 3-itemset candidates that contain infrequent 2-itemsets

Illustrating Apriori Principle

Item	Count
Bread	4
Coke	2
Milk	4
Juice	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread,Milk}	3
{Juice, Bread}	2
{Bread,Diaper}	3
{Juice,Milk}	2
{Diaper,Milk}	3
{Juice,Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 4 = 16$$

$$6 + 6 + 1 = 13$$



Triplets (3-itemsets)

Itemset	Count
{Juice, Diaper, Milk}	2
{Juice, Bread, Diaper}	2
{Bread, Diaper, Milk}	2
{Juice, Bread, Milk}	1

Eliminate infrequent 3-itemset candidates

Apriori Algorithm: Frequent Itemsets Generation

F_k : frequent ***k***-itemsets

L_k : candidate ***k***-itemsets

- Let ***k***=1
- Generate $F_1 = \{\text{frequent 1-itemsets}\}$
- **Repeat until F_k is empty**

■ Candidate Generation:

Generate L_{k+1} from F_k

■ Candidate Pruning:

Prune candidate itemsets in L_{k+1} containing subsets of length ***k*** that are infrequent

■ Support Counting:

Count the support of each candidate in L_{k+1} by scanning the DB

■ Candidate Elimination:

Eliminate candidates in L_{k+1} that are infrequent, leaving only those that are frequent $\Rightarrow F_{k+1}$

Apriori Algorithm: Frequent Itemsets Generation

The upcoming slides will discuss the following steps in the Apriori Algorithm:

- **Candidate Generation:** Generate candidate itemsets.
- **Candidate Pruning:** Prune candidate itemsets that contain infrequent subsets.
- **Support Counting:** Count the support of each candidate itemset.
- **Candidate Elimination:** Eliminate candidates that are infrequent.

We will go through each of these steps in detail.

Candidate Generation

- **Brute-Force Method**

- Exploring all possible combinations of itemsets to generate candidates.
- Computationally intensive, especially for large datasets.

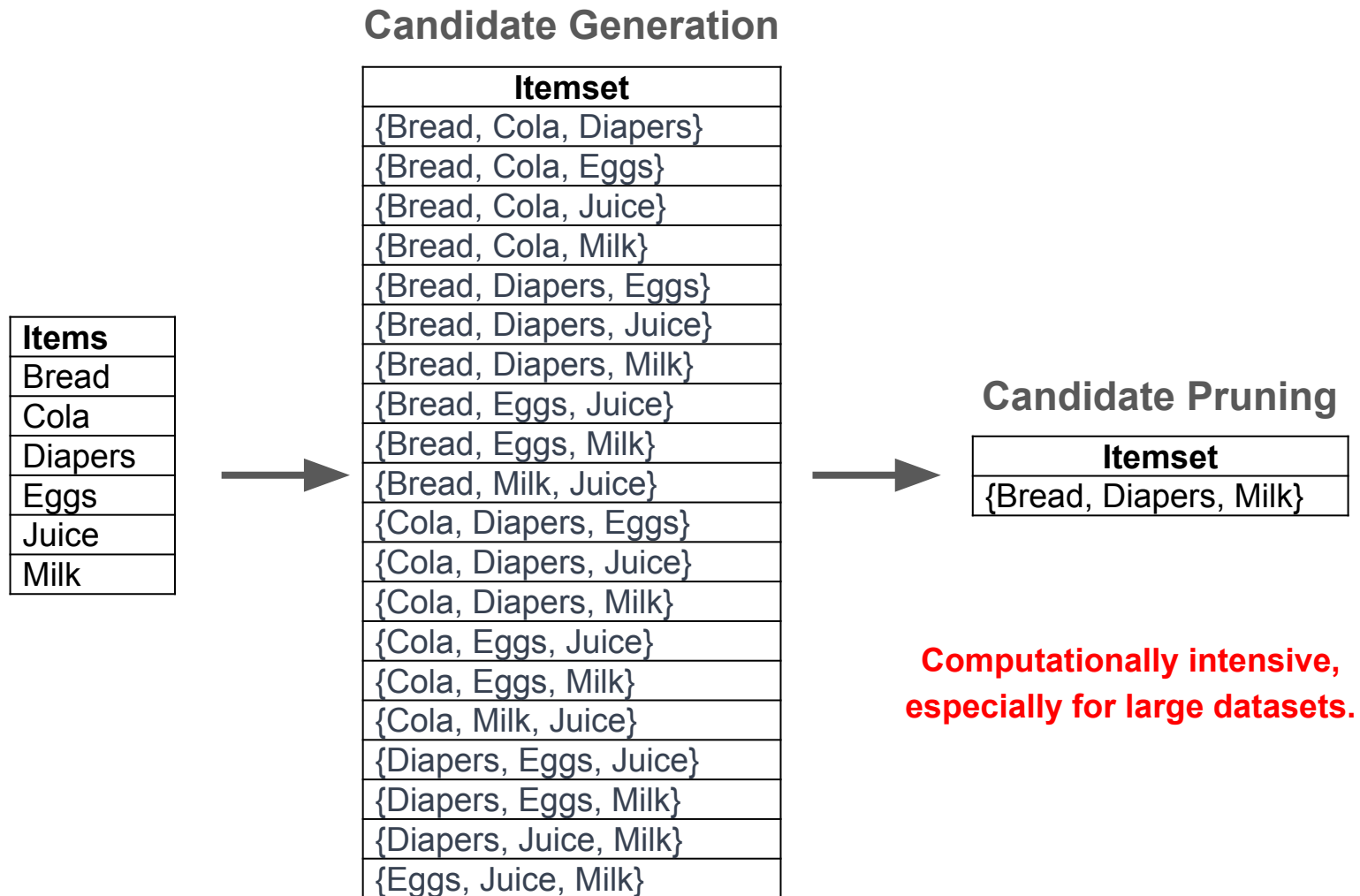
- **Merge Fk-1 and F1 Itemsets**

- Combining **frequent** (k-1)-itemsets with individual **frequent** items to form candidates.
- Example: If {A, B} is frequent, combining it with {C} to create {A, B, C}.

- **Merge Fk-1 and Fk-1 Itemsets**

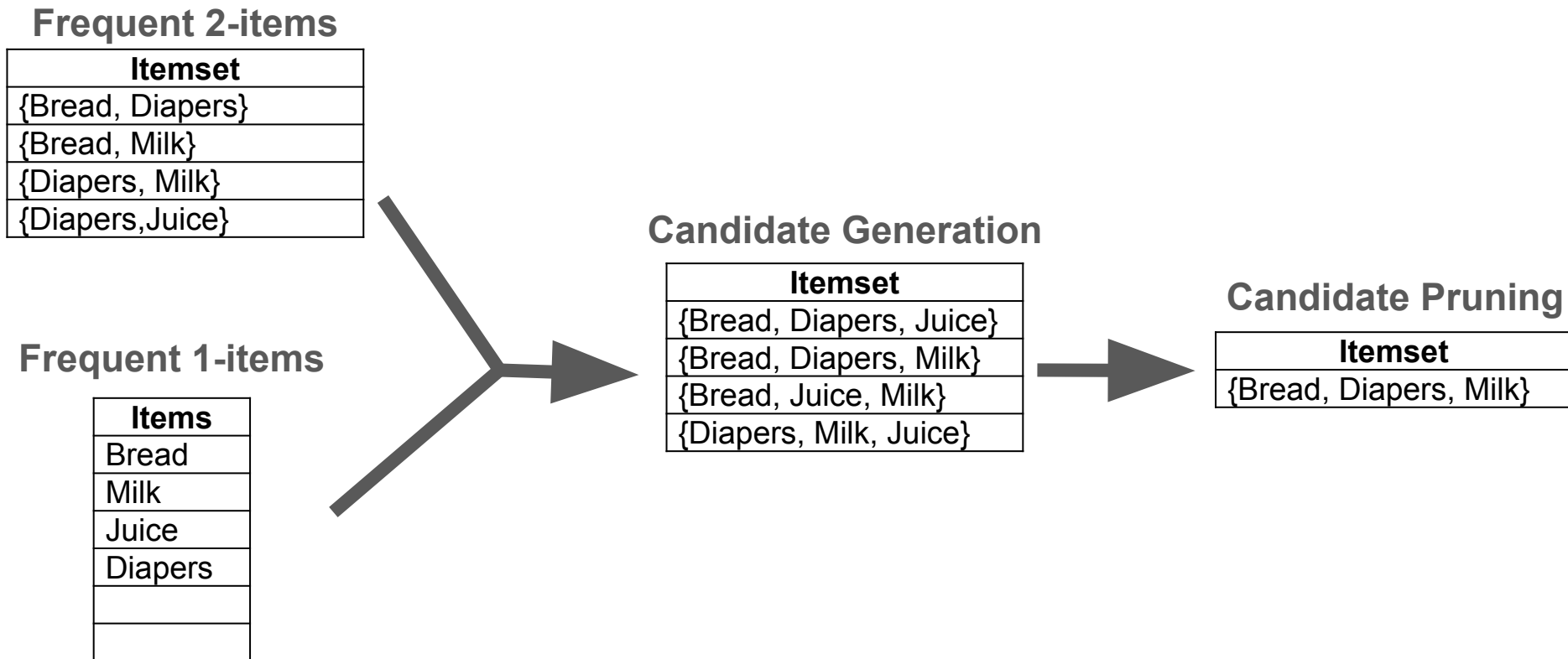
- Merging **frequent** (k-1)-itemsets with other **frequent** (k-1)-itemsets to form candidates.
- Example: Merging {A, B} and {C, D} to create {A, B, C, D}.

Candidate Generation: Brute-force method



A brute-force method for generating candidate 3-itemsets: ${}^6C_3 = 20$

Candidate Generation: Merge Fk-1 and F1 itemsets



Generate and prune candidate k -itemsets by merging a frequent $(k-1)$ itemset with a frequent item. Note that some of the candidate are unnecessary because their subsets are infrequent.

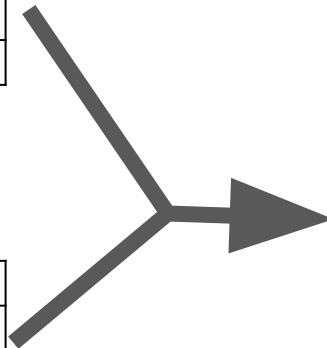
Candidate Generation: Merge Fk-1 and Fk-1 itemsets

Frequent 2-items

Itemset
{Bread, Diapers}
{Bread, Milk}
{Diapers, Milk}
{Diapers, Juice}

Frequent 2-items

Itemset
{Bread, Diapers}
{Bread, Milk}
{Diapers, Milk}
{Diapers, Juice}



Candidate Generation

Itemset
{Bread, Diapers, Milk}



Candidate Pruning

Itemset
{Bread, Diapers, Milk}

Generate and prune candidate k -itemsets by merging pairs of frequent $(k-1)$ -itemsets.

Alternate $F_{k-1} \times F_{k-1}$ Prefix Method

- Merge two frequent $(k-1)$ -itemsets if their first $(k-2)$ items are identical.
- $F_3 = \{ABC, ABD, ABE, ACD, BCD, BDE, CDE\}$
 - Merge(ABC, ABD) = ABCD
 - Merge(ABC, ABE) = ABCE
 - Merge(ABD, ABE) = ABDE
 - Do not merge(ABD, ACD) because they share only **prefix** of length 1 instead of length 2.
- The main advantages of this method:
 - avoids generating redundant candidates.
 - generates only valid candidates.

Candidate Pruning

- Let $F_3 = \{ABC, ABD, ABE, ACD, BCD, BDE, CDE\}$ be the set of frequent 3-itemsets.
- $L_4 = \{ABCD, ABCE, ABDE\}$ is the set of generated candidate 4-itemsets.
- Candidate pruning
 - Prune ABCE because ACE and BCE are infrequent
 - Prune ABDE because ADE is infrequent
- After candidate pruning: $L_4 = \{ABCD\}$

Candidate pruning allows the removal of some itemsets without calculating the support, based on the anti-monotonic property.

Illustrating Apriori Principle

Item	Count
Bread	4
Coke	2
Milk	4
Juice	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread,Milk}	3
{Bread,Juice}	2
{Bread,Diaper}	3
{Milk,Juice}	2
{Milk,Diaper}	3
{Juice,Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Minimum Support = 3

If every subset is considered,

$${}^6C_1 + {}^6C_2 + {}^6C_3 \\ 6 + 15 + 20 = 41$$

With support-based pruning,

$$6 + 6 + 1 = 13$$



Itemset	Count
{Bread, Diaper, Milk}	2

Triplets (3-itemsets)

Use of $F_{k-1} \times F_{k-1}$ method for candidate generation results in only one 3-itemset.
This is eliminated after the support counting step.

Alternate $F_{k-1} \times F_{k-1}$ Method

- Merge two frequent $(k-1)$ -itemsets if the last $(k-2)$ items of the first one is identical to the first $(k-2)$ items of the second.
- $F_3 = \{ABC, ABD, ABE, ACD, BCD, BDE, CDE\}$
 - Merge(ABC, BCD) = ABCD
 - Merge(ABD, BDE) = ABDE
 - Merge(ACD, CDE) = ACDE
 - Merge(BCD, CDE) = BCDE
- This variant shifts from strict prefix alignment to a broader "overlapping region" alignment.

Candidate Pruning for Alternate $F_{k-1} \times F_{k-1}$ Method

- Let $F_3 = \{ABC, ABD, ABE, ACD, BCD, BDE, CDE\}$ be the set of frequent 3-itemsets.
- $L_4 = \{ABCD, ABDE, ACDE, BCDE\}$ is the set of generated candidate 4-itemsets.
- Candidate pruning
 - Prune ABDE because ADE is infrequent
 - Prune ACDE because ACE and ADE are infrequent
 - Prune BCDE because BCE
- After candidate pruning: $L_4 = \{ABCD\}$